🚩 **Writeup**
Written by: Shadman Zaman Sajid

Challenge: WaF
Category: Web /

Statement:

You can't get the /flag.txt ever.
Link : http://45.56.66.96:7789/

1. Reconnaissance

I started by accessing the provided link: **http://45.56.66.96:7789/.** The page presented a simple form asking for a name, but as usual, I immediately checked the page source (Ctrl+U).

I found a crucial piece of commented-out backend code that revealed how the server handles requests:

Python

```
# @app.after_request
# def index(filename: str = "index.html"):
#     if ".." in filename or "%" in filename:
#         return "No no not like that :("
```

2. Vulnerability Analysis

The code snippet gave me the blueprint for the exploit:

The Injection Point:
The server accepts a filename argument (likely via the URL path).

The Objective:
The hint "You can't get the /flag.txt ever" told me the target file is at the root: /flag.txt.

The Constraints:

The server implements a strict filter.

It blocks .. (preventing standard Directory Traversal like ../../).

It blocks % (preventing URL encoding bypasses like %2e%2e).

I tried accessing http://45.56.66.96:7789/flag.txt directly, but it failed. I realized the application is likely running inside a subdirectory (e.g., /app/src/), so I needed to traverse up to the root directory.

3. The Bypass Strategy

Since standard Python path traversal was blocked, I suspected the backend might be passing this filename to a shell command like curl or wget to fetch the file content.

If the backend uses curl, I could exploit Sequence/Globbing features. In curl, curly braces {} are used for pattern matching. Crucially:

{.} is interpreted as a literal .

Therefore, {.}{.} is interpreted by curl as ..

The Logic: The Python filter if ".." in filename checks the string literally.

My input: {.}{.}

Python sees: { followed by . followed by }... It does NOT see two dots together. The check passes.

The Backend (Curl) sees: .. (after resolving the glob patterns). The traversal happens.

 # 4. The Exploit
I constructed a payload to move up two directories and grab the flag.

Payload: /{.}{.}/{.}{.}/flag.txt

Plaintext

http://45.56.66.96:7789/{.}{.}/{.}{.}/flag.txt

and boom!! the site contains nothing but the flag

Flag: KCTF{7fdbbcd6c3cee0ae65c5ca327c14a25f6e473d1c}

5. Conclusion

Sending this request bypassed the filter and allowed the backend to traverse to the root directory. The server returned the content of /flag.txt.